

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Database Management Systems Lab

Project Phase: 2

Online Movie Ticket Booking Management System

Group Members:

Sr. No.	Name	PRN	Branch	Division
1	Parvez Khan	24070122133	CS	B2
2	Rudra Patel	24070122134	CS	B2
3	Prachi Jagota	24070122138	CS	B2
4	Prisha Gupta	24070122145	CS	B2

Under the Guidance of

Dr. Sudhanshu Gonge

Department of Computer Science
Symbiosis Institute of Technology, Pune

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Certificate of Approval

This is to certify that the project entitled “**Online Movie Ticket Booking Management System**” has been successfully completed by the following group members:

- Parwez Khan (PRN: 24070122133)
- Rudra Patel (PRN: 24070122134)
- Prachi Jagota (PRN: 24070122138)
- Prisha Gupta (PRN: 24070122145)

This project was carried out under the guidance of Dr. Sudhanshu Gonge in the **Department of Computer Science** at the **Symbiosis Institute of Technology, Pune**.

Dr. Sudhanshu Gonge

Project Guide

Department of Computer Science
Pune

Head of Department

Department of Computer Science

Symbiosis Institute of Technology,

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Title of the project:

Online Movie Ticket Booking Management System

About the project:

The Online Movie Ticket Booking Management System is a database-driven application designed to simplify and automate the process of booking movie tickets. Developed using Java for the application layer and MySQL for database management, this system provides a comprehensive solution for both customers and administrators. Through this system, customers can register, browse available movies and shows, select seats, and make payments using multiple modes. All booking and transaction information is stored in a centralized database, ensuring data accuracy and consistency.

The system also supports cancellations and automated refund processing, maintaining complete records for every transaction. Administrators have full control over managing movies, screens, shows, and viewing all bookings. The database design employs foreign key constraints, stored procedures, triggers, and functions to maintain data integrity and automate key operations. With a well-structured and normalized schema, the system ensures efficient data retrieval and management.

Addressed pain points:

The pain point addressed by this system lies in the inefficiency of manual or poorly organized ticket booking methods. Customers face issues like double seat booking, limited real-time seat visibility, and unreliable payment and cancellation handling.

This system solves these problems by providing automated seat management, real-time show availability, multiple payment options, and systematic cancellation and refund processing. By centralizing all data in a structured MySQL database, the system eliminates data inconsistency and enables both customers and administrators to operate efficiently.

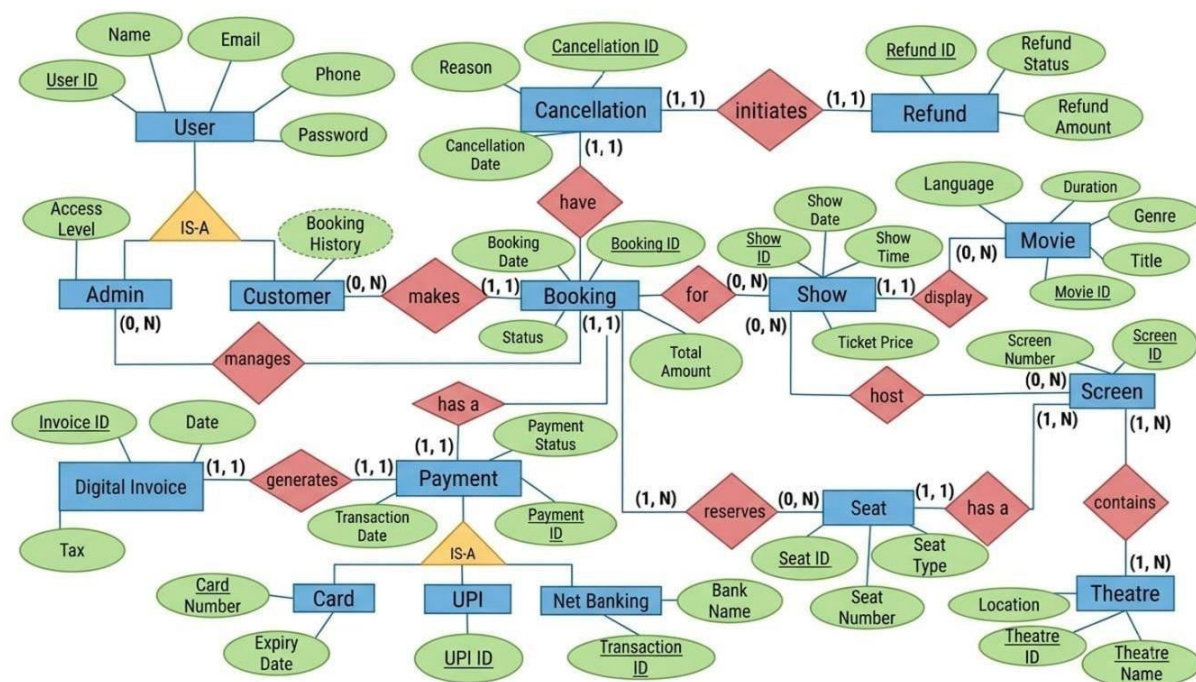
SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Extended Entity-Relationship Diagram for Online Movie Booking System



SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Anomalies:

A database anomaly is a fault within a database, which can occur because of poor planning or when everything is stored in a flat file. These anomalies naturally occur and result in data that does not match the real-world the database purports to represent.

Anomalies occurring in our database:

1. 1NF Anomalies:

Insert Anomaly: In the Booking table, before normalization, if we tried to store a user's booking along with show and movie details in the same row, we would need to repeat all movie and show information for each seat booked. This creates redundant data and makes insertion complex.

Update Anomaly: If a movie's title or duration is stored redundantly alongside booking records, updating the movie's title would require updating every booking row associated with that movie, leading to inconsistency if some rows are missed.

Delete Anomaly: If we delete a booking, we may inadvertently lose information about the show and movie if those details were only stored within that booking record and not maintained in separate tables.

Table 1. Example of unnormalized Booking table (before 1NF):

	BookingID	UserName	MovieTitle	ShowDate	Seats	Amount
▶	1	Rahul	Interstellar	2026-04-01	A1,A2	500.00
	2	Priya	Interstellar	2026-04-01	B1	250.00

2. 2NF Anomalies:

Insert Anomaly: To insert a new show, we must provide a valid MovieID and ScreenID. Show details cannot be stored independently without first having a corresponding movie and screen record, creating dependency issues.

Update Anomaly: If movie information like Genre or Language is stored directly in the ShowTable instead of a separate Movie table, updating that information would require changes across all show records for that movie.

Delete Anomaly: Deleting a show that has associated bookings creates a foreign key violation. If bookings reference a show that no longer exists, data integrity is compromised.

Table 2. Example showing partial dependency issue (before 2NF):

	ShowID	MovieID	MovieTitle	Genre	ShowDate
▶	1	1	Interstellar	Sci-Fi	2026-04-01
	2	1	Interstellar	Sci-Fi	2026-04-02
	3	2	Inception	Thriller	2026-04-03

3. 3NF Anomalies:

Insert Anomaly: Inserting a new payment record requires providing both the BookingID and UserID. If UserID is stored in the Payment table as well, it creates a transitive dependency since UserID can be derived from BookingID through the Booking table.

Update Anomaly: If theatre name and location are stored in the Screen table instead of a separate Theatre table, updating a theatre's location would require updating every screen record belonging to that theatre.

Delete Anomaly: Deleting a payment record that has an associated DigitalInvoice will break the foreign key reference in the DigitalInvoice table, causing data inconsistency.

Table 3. Example showing transitive dependency (before 3NF):

	PaymentID	BookingID	UserEmail	Mode
▶	1	1	rahul@gmail.com	UPI
	2	2	rahul@gmail.com	CARD
	3	3	priya@gmail.com	CASH

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Functional Dependencies:

Functional dependency refers to a relationship that is present between attributes of any table that are dependent on each other. It helps in avoiding data redundancy and identifying bad design.

Functional dependencies of schema occurring in our database:

4. User Table:

- UserID → Name, Email, Phone, Password, Role
- Email → UserID, Name, Phone, Password, Role

5. Movie Table:

- MovieID → Title, Genre, Duration, Language

6. Theatre Table:

- TheatreID → Name, Location

7. Screen Table:

- ScreenID → ScreenNumber, TheatreID
- (TheatreID, ScreenNumber) → ScreenID

8. Seat Table:

- SeatID → SeatNumber, SeatType, ScreenID, Status
- (ScreenID, SeatNumber) → SeatID

9. ShowTable:

- ShowID → ShowDate, ShowTime, Price, MovieID, ScreenID
- (ScreenID, ShowDate, ShowTime) → ShowID

10. Booking Table:

- BookingID → BookingDate, TotalAmount, UserID, ShowID

11. Payment Table:

- PaymentID → Mode, Status, TransactionDate, BookingID
- BookingID → PaymentID

12. DigitalInvoice Table:

- InvoiceID -> InvoiceDate, TaxAmount, PaymentID
- PaymentID -> InvoiceID

13. Cancellation Table:

- CancellationID -> CancellationDate, Reason, RefundAmount, BookingID
- BookingID -> CancellationID

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Normalization:

A normalization procedure, which combines and splits tables, is usually sufficient to remove anomalies. By normalizing the database, we reduce the likelihood of creating tables that generate anomalies. Normalization is necessary because without it, the overall integrity of the data stored in the database will eventually degrade due to data anomalies.

Types of Normalization:

1. Unnormalized Form or UNF:

It is the simplest database model, also known as non-first normal form (NF2). A UNF model will suffer problems like data redundancy and lacks the efficiency of database normalization.

2. First Normal Form or 1NF:

A relation is in first normal form only if the relational table does not contain any multi-valued attribute but contains only single-valued (atomic) attributes.

3. Second Normal Form or 2NF:

A relation is in second normal form if:

- It is in first normal form (1NF).
- It does not contain any partial dependencies. No non-prime attribute is functionally dependent on any proper subset of the candidate key.

4. Third Normal Form or 3NF:

A relation R is in 3NF if for every non-trivial functional dependency $X \rightarrow Y$ over R:

- R should be in 2NF.
- X should be a candidate key or superkey, OR
- Y should be a prime attribute.

Normalization occurring in our database:

1. 1NF

In the unnormalized version, the Booking table contained multi-valued attributes where multiple seat numbers were stored for a single booking in one row. To adhere to 1NF, each booking record now contains atomic values. The Seat table is maintained separately with individual SeatID records, each with a single SeatNumber and SeatType, ensuring all attributes hold atomic values.

Table 4. Before 1NF (unnormalized - multiple seats per row):

	BookingID	UserName	MovieTitle	ShowDate	Seats	Amount
▶	1	Rahul	Interstellar	2026-04-01	A1,A2	500.00
	2	Priya	Interstellar	2026-04-01	B1	250.00

Table 5. After 1NF (each seat as separate record in Seat table):

	BookingID	UserName	MovieTitle	ShowDate	SeatNumber	Amount
▶	1	Rahul	Interstellar	2026-04-01	A1	250.00
	1	Rahul	Interstellar	2026-04-01	A2	250.00
	2	Priya	Interstellar	2026-04-01	B1	250.00

2. 2NF

The ShowTable previously had movie attributes like Title stored directly within it. These attributes (Title, Genre, Duration, Language) are functionally dependent only on MovieID and not on ShowID. This is a partial dependency. To achieve 2NF, all movie-related attributes were separated into a dedicated Movie table with MovieID as the primary key, and ShowTable now only stores ShowID as the primary key with MovieID as a foreign key.

Table 6. Before 2NF (movie details repeated in ShowTable):

	ShowID	MovieID	MovieTitle	Genre	ShowDate
▶	1	1	Interstellar	Sci-Fi	2026-04-01
	2	1	Interstellar	Sci-Fi	2026-04-02
	3	2	Inception	Thriller	2026-04-03

Table 7. After 2NF (separated into Movie and ShowTable): Table

7.1

	MovieID	Title	Genre
▶	1	Interstellar	Sci-Fi
	2	Inception	Thriller
•	NULL	NULL	NULL

Table 7.2

	ShowID	MovieID	ShowDate
▶	1	1	2026-04-01
	2	1	2026-04-02
	3	2	2026-04-03
•	NULL	NULL	NULL

3. 3NF

In the Payment table, if userEmail had been stored directly, it would depend on UserID which is derived from BookingID, creating a transitive dependency (PaymentID → BookingID → UserID → userEmail). To achieve 3NF, all user-related attributes remain only in the User table. The Payment table references BookingID as a foreign key, and user details are retrieved via JOIN operations. All tables in our final schema are in 3NF as no non-key attribute transitively depends on the primary key.

Table 8. Before 3NF

	PaymentID	userID	BookingID	Email	Mode
▶	1	2	1	abc@gmail.com	UPI
	2	3	2	xyz@gmail.com	CARD
	3	4	3	rudra123@mail.com	UPI
	4	3	4	xyz@gmail.com	UPI
	5	2	1	abc@gmail.com	UPI

Table 9. After 3NF

	PaymentID	Mode	Status	TransactionDate	BookingID
▶	1	UPI	SUCCESS	2026-03-30	1
	2	CARD	SUCCESS	2026-03-30	2
	3	UPI	SUCCESS	2026-03-30	3
	4	UPI	SUCCESS	2026-03-31	4
	5	UPI	SUCCESS	2026-03-31	1
*	NULL	NULL	NULL	NULL	NULL

	userID	email
▶	1	admin@cinebook.com
	2	abc@gmail.com
	3	xyz@gmail.com
	4	rudra123@mail.com
	5	rahul@gmail.com

Table 10. Final normalized schema summary:

Table	Primary Key	Normal Form	Status
User	UserID	3NF	No anomalies
Movie	MovieID	3NF	No anomalies
Theatre	TheatreID	3NF	No anomalies
Screen	ScreenID	3NF	No anomalies
Seat	SeatID	3NF	No anomalies
ShowTable	ShowID	3NF	No anomalies
Booking	BookingID	3NF	No anomalies
Payment	PaymentID	3NF	No anomalies
DigitalInvoice	InvoiceID	3NF	No anomalies
Cancellation	CancellationID	3NF	No anomalies

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

What is MySQL:

MySQL is a relational database management system (RDBMS) developed by Oracle that is based on structured query language (SQL). It is widely used for web applications and is known for its reliability, scalability, and ease of use.

Implementation of the tables in MySQL:

```
CREATE DATABASE movie_db;
USE movie_db;

CREATE TABLE User (
    UserID    INT AUTO_INCREMENT PRIMARY KEY,
    Name      VARCHAR(50),
    Email     VARCHAR(50) UNIQUE,
    Phone     VARCHAR(15),
    Password  VARCHAR(50),
    Role      VARCHAR(10)
);

CREATE TABLE Movie (
    MovieID   INT AUTO_INCREMENT PRIMARY KEY,
    Title     VARCHAR(100),
    Genre     VARCHAR(50),
    Duration  INT,
    Language  VARCHAR(30)
);

CREATE TABLE Theatre (
    TheatreID INT AUTO_INCREMENT PRIMARY KEY,
    Name      VARCHAR(100),
    Location  VARCHAR(100)
);

CREATE TABLE Screen (
    ScreenID   INT AUTO_INCREMENT PRIMARY KEY,
    ScreenNumber INT,
    TheatreID  INT,
    FOREIGN KEY (TheatreID) REFERENCES Theatre(TheatreID)
);

CREATE TABLE Seat (
    SeatID     INT AUTO_INCREMENT PRIMARY KEY,
    SeatNumber VARCHAR(10),
    SeatType   VARCHAR(20),
    ScreenID   INT,
    Status     VARCHAR(20) DEFAULT 'AVAILABLE',
```

```
    FOREIGN KEY (ScreenID) REFERENCES  
Screen(ScreenID) );
```

```
CREATE TABLE ShowTable (  
    ShowID    INT AUTO_INCREMENT PRIMARY KEY,  
    ShowDate  DATE,  
    ShowTime  TIME,  
    Price     DECIMAL(8,2),  
    MovieID   INT,  
    ScreenID  INT,  
    FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),  
    FOREIGN KEY (ScreenID) REFERENCES Screen(ScreenID)  
);
```

```
CREATE TABLE Booking (  
    BookingID  INT AUTO_INCREMENT PRIMARY KEY,  
    BookingDate DATE,  
    TotalAmount DECIMAL(10,2),  
    UserID     INT,  
    ShowID     INT,  
    FOREIGN KEY (UserID) REFERENCES User(UserID),  
    FOREIGN KEY (ShowID) REFERENCES ShowTable(ShowID)  
);
```

```
CREATE TABLE Payment (  
    PaymentID    INT AUTO_INCREMENT PRIMARY KEY,  
    Mode          VARCHAR(20),  
    Status        VARCHAR(20),  
    TransactionDate DATE,  
    BookingID     INT,  
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)  
);
```

```
CREATE TABLE DigitalInvoice (  
    InvoiceID    INT AUTO_INCREMENT PRIMARY KEY,  
    InvoiceDate  DATE,  
    TaxAmount   DECIMAL(8,2),  
    PaymentID   INT,  
    FOREIGN KEY (PaymentID) REFERENCES Payment(PaymentID)  
);
```

```
CREATE TABLE Cancellation (  
    CancellationID  INT AUTO_INCREMENT PRIMARY KEY,  
    CancellationDate DATE,  
    Reason          VARCHAR(200),  
    RefundAmount    DECIMAL(10,2),  
    BookingID       INT,  
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)  
);
```

```
# functions
```

```
DELIMITER $$  
CREATE FUNCTION GetRefundAmount(p_BookingID INT)  
RETURNS DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    DECLARE v_Amount DECIMAL(10,2);  
    SELECT TotalAmount INTO v_Amount  
    FROM Booking
```

```
WHERE BookingID = p_BookingID;
RETURN v_Amount * 0.80;
END $$
DELIMITER ;
```

```
DELIMITER $$
CREATE FUNCTION GetUserBookingCount(p_UserID INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_Count INT;
    SELECT COUNT(*) INTO v_Count
    FROM Booking
    WHERE UserID = p_UserID;
    RETURN v_Count;
END $$
DELIMITER ;
```

```
DELIMITER $$
CREATE FUNCTION GetTaxAmount(p_BookingID INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_Amount DECIMAL(10,2);
    SELECT TotalAmount INTO v_Amount
    FROM Booking WHERE BookingID = p_BookingID;
    RETURN ROUND(v_Amount * 0.18, 2);
END $$
DELIMITER ;
```

stored procedures

```
DELIMITER $$
CREATE PROCEDURE RegisterCustomer(
    IN p_Name    VARCHAR(50),
    IN p_Email   VARCHAR(50),
    IN p_Phone   VARCHAR(15),
    IN p_Password VARCHAR(50)
)
BEGIN
    IF EXISTS (SELECT 1 FROM User WHERE Email = p_Email) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Email already registered.';
    ELSE
        INSERT INTO User(Name, Email, Phone, Password, Role)
        VALUES (p_Name, p_Email, p_Phone, p_Password, 'CUSTOMER');
        SELECT LAST_INSERT_ID() AS NewUserID;
    END IF;
END $$
DELIMITER ;
```

```
DELIMITER $$
CREATE PROCEDURE BookTicket(
    IN p_UserID    INT,
    IN p_ShowID    INT,
    IN p_TotalAmount DECIMAL(10,2),
    IN p_PaymentMode VARCHAR(20)
)
BEGIN
    DECLARE v_BookingID INT;
```

```

DECLARE v_PaymentID INT;

INSERT INTO Booking(BookingDate, TotalAmount, UserID, ShowID)
VALUES (CURDATE(), p_TotalAmount, p_UserID, p_ShowID);
SET v_BookingID = LAST_INSERT_ID();

INSERT INTO Payment(Mode, Status, TransactionDate, BookingID)
VALUES (p_PaymentMode, 'SUCCESS', CURDATE(), v_BookingID);
SET v_PaymentID = LAST_INSERT_ID();
INSERT INTO DigitalInvoice(InvoiceDate, TaxAmount, PaymentID)
VALUES (CURDATE(), ROUND(p_TotalAmount * 0.18, 2), v_PaymentID);

SELECT v_BookingID AS BookingID, v_PaymentID AS PaymentID;
END $$
DELIMITER ;

DELIMITER $$
CREATE PROCEDURE CancelBooking(
    IN p_BookingID INT,
    IN p_Reason    VARCHAR(200)
)
BEGIN
    DECLARE v_Refund DECIMAL(10,2);
    SET v_Refund = GetRefundAmount(p_BookingID);

    INSERT INTO Cancellation(
        CancellationDate, Reason, RefundAmount, BookingID)
    VALUES (CURDATE(), p_Reason, v_Refund, p_BookingID);

    SELECT v_Refund AS RefundAmount;
END $$
DELIMITER ;

# trigger
DELIMITER $$
CREATE TRIGGER trg_AfterBookingInsert
AFTER INSERT ON Booking
FOR EACH ROW
BEGIN
    UPDATE Seat
    SET Status = 'BOOKED'
    WHERE ScreenID = (
        SELECT ScreenID FROM ShowTable WHERE ShowID = NEW.ShowID
    )
    AND Status = 'AVAILABLE'
    LIMIT 1;
END $$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER trg_BeforeBookingInsert
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    DECLARE v_Available INT;

    SELECT COUNT(*) INTO v_Available
    FROM Seat
    WHERE ScreenID = (
        SELECT ScreenID FROM ShowTable WHERE ShowID = NEW.ShowID

```

```

)
AND Status = 'AVAILABLE';

IF v_Available = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'No seats available for this show.';
END IF;
END $$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER trg_AfterPaymentInsert
AFTER INSERT ON Payment
FOR EACH ROW
BEGIN
    IF NEW.Status = 'SUCCESS' THEN
        INSERT INTO DigitalInvoice(InvoiceDate, TaxAmount, PaymentID)
        SELECT CURDATE(),
            ROUND(b.TotalAmount * 0.18, 2),
            NEW.PaymentID
        FROM Booking b
        WHERE b.BookingID = NEW.BookingID;
    END IF;
END $$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER trg_AfterCancellationInsert
AFTER INSERT ON Cancellation
FOR EACH ROW
BEGIN
    UPDATE Seat
    SET Status = 'AVAILABLE'
    WHERE ScreenID = (
        SELECT s.ScreenID
        FROM Booking b
        JOIN ShowTable s ON b.ShowID = s.ShowID
        WHERE b.BookingID = NEW.BookingID
    )
    AND Status = 'BOOKED'
    LIMIT 1;
END $$
DELIMITER ;

```

#Sample INSERT statements:

```

INSERT INTO User (Name, Email, Phone, Password, Role) VALUES
('Admin', 'admin@cinebook.com', '9999999999', 'admin@123', 'ADMIN'),
('abc', 'abc@gmail.com', '9876543210', 'abc123', 'CUSTOMER'),
('xyz', 'xyz@gmail.com', '9123456780', 'xyz123', 'CUSTOMER');
INSERT INTO Movie (Title, Genre, Duration, Language) VALUES
('Interstellar', 'Sci-Fi', 169, 'English'),
('Inception', 'Thriller', 148, 'English'),
('KGF Chapter 2', 'Action', 168, 'Kannada');
INSERT INTO Theatre (Name, Location) VALUES
('PVR Cinemas', 'Pune'),
('INOX', 'Mumbai');

INSERT INTO Screen (ScreenNumber, TheatreID) VALUES

```



```
(1, 1),  
(2, 1),  
(1, 2);
```

```
INSERT INTO Seat (SeatNumber, SeatType, ScreenID, Status) VALUES  
( 'A1', 'STANDARD', 1, 'AVAILABLE'),  
( 'A2', 'STANDARD', 1, 'AVAILABLE'),  
( 'A3', 'STANDARD', 1, 'AVAILABLE'),  
( 'B1', 'PREMIUM', 1, 'AVAILABLE'),  
( 'B2', 'PREMIUM', 1, 'AVAILABLE'),  
  
( 'A1', 'STANDARD', 2, 'AVAILABLE'),  
( 'A2', 'STANDARD', 2, 'AVAILABLE'),  
  
( 'A1', 'STANDARD', 3, 'AVAILABLE');
```

```
INSERT INTO ShowTable (ShowDate, ShowTime, Price, MovieID, ScreenID) VALUES  
( '2026-04-01', '18:00:00', 250.00, 1, 1),  
( '2026-04-01', '21:00:00', 300.00, 2, 2),  
( '2026-04-02', '20:00:00', 280.00, 3, 3);
```

```
INSERT INTO Booking (BookingDate, TotalAmount, UserID, ShowID) VALUES  
(CURDATE(), 500.00, 2, 1),  
(CURDATE(), 300.00, 3, 2);
```

```
INSERT INTO Payment (Mode, Status, TransactionDate, BookingID) VALUES  
( 'UPI', 'SUCCESS', CURDATE(), 1),  
( 'CARD', 'SUCCESS', CURDATE(), 2);
```

```
INSERT INTO DigitalInvoice (InvoiceDate, TaxAmount, PaymentID) VALUES (CURDATE(),  
90.00, 1),  
(CURDATE(), 54.00, 2);
```

```
INSERT INTO Cancellation (CancellationDate, Reason, RefundAmount, BookingID)  
VALUES  
(CURDATE(), 'Change of plans', 400.00, 1);
```

Table outputs:

1. User

Stores details of users including personal info, login credentials, and role (admin/customer).

	UserID	Name	Email	Phone	Password	Role
▶	1	Admin	admin@cinebook.com	9999999999	admin@123	ADMIN
	2	abc	abc@gmail.com	9876543210	abc123	CUSTOMER
	3	xyz	xyz@gmail.com	9123456780	xyz123	CUSTOMER
	4	Rudra Patel	rudra123@mail.com	9988776655	rudra@123	CUSTOMER
•	NULL	NULL	NULL	NULL	NULL	NULL

2. Movie

Contains information about movies such as title, genre, duration, and language.

	MovieID	Title	Genre	Duration	Language
▶	1	Interstellar	Sci-Fi	169	English
	2	Inception	Thriller	148	English
	3	KGF Chapter 2	Action	168	Kannada
•	NULL	NULL	NULL	NULL	NULL

3. Theatre

Holds details of theatres including name and location.

	TheatreID	Name	Location
▶	1	PVR Cinemas	Pune
	2	INOX	Mumbai
•	NULL	NULL	NULL

4. Screen

Represents individual screens within a theatre, linked to a specific theatre.

	ScreenID	ScreenNumber	TheatreID
▶	1	1	1
	2	2	1
	3	1	2
•	NULL	NULL	NULL

5. Seat

Stores seat details for each screen including seat number, type, and availability status.

	SeatID	SeatNumber	SeatType	ScreenID	Status
▶	1	A1	STANDARD	1	AVAILABLE
	2	A2	STANDARD	1	AVAILABLE
	3	A3	STANDARD	1	AVAILABLE
	4	B1	PREMIUM	1	AVAILABLE
	5	B2	PREMIUM	1	AVAILABLE
	6	A1	STANDARD	2	AVAILABLE
	7	A2	STANDARD	2	AVAILABLE
	8	A1	STANDARD	3	AVAILABLE
•	NULL	NULL	NULL	NULL	NULL

6. ShowTable

Contains show schedule details like date, time, price, and associated movie and screen.

	ShowID	ShowDate	ShowTime	Price	MovieID	ScreenID
▶	1	2026-04-01	18:00:00	250.00	1	1
	2	2026-04-01	21:00:00	300.00	2	2
	3	2026-04-02	20:00:00	280.00	3	3
•	NULL	NULL	NULL	NULL	NULL	NULL

7. Booking

Records ticket bookings made by users for specific shows along with total amount.

	BookingID	BookingDate	TotalAmount	UserID	ShowID
▶	1	2026-03-30	500.00	2	1
	2	2026-03-30	300.00	3	2
	3	2026-03-30	560.00	4	3
•	NULL	NULL	NULL	NULL	NULL

8. Payment

Stores payment details for bookings including mode, status, and transaction date.

	PaymentID	Mode	Status	TransactionDate	BookingID
▶	1	UPI	SUCCESS	2026-03-30	1
	2	CARD	SUCCESS	2026-03-30	2
	3	UPI	SUCCESS	2026-03-30	3
•	NULL	NULL	NULL	NULL	NULL

9. DigitalInvoice

Keeps invoice records generated for successful payments with tax details.

	InvoiceID	InvoiceDate	TaxAmount	PaymentID
▶	1	2026-03-30	90.00	1
	2	2026-03-30	54.00	2
	3	2026-03-30	100.80	3
•	NULL	NULL	NULL	NULL

10. Cancellation

Stores information about cancelled bookings including reason and refund amount.

	CancellationID	CancellationDate	Reason	RefundAmount	BookingID
▶	1	2026-03-30	Change of plans	400.00	1
	2	2026-03-30	choose wrong movie	448.00	3
•	NULL	NULL	NULL	NULL	NULL

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Queries Implemented:

Query 1: List all movies with their details.

```
SELECT MovieID, Title, Genre, Language, Duration FROM Movie  
ORDER BY Title;
```

	MovieID	Title	Genre	Language	Duration
▶	2	Inception	Thriller	English	148
	1	Interstellar	Sci-Fi	English	169
	3	KGF Chapter 2	Action	Kannada	168
★	NULL	NULL	NULL	NULL	NULL

Query 2: Find all available seats for a given screen.

```
SELECT SeatID, SeatNumber, SeatType  
FROM Seat  
WHERE ScreenID = 1 AND Status = 'AVAILABLE';
```

	SeatID	SeatNumber	SeatType
▶	1	A1	STANDARD
	2	A2	STANDARD
	3	A3	STANDARD
	4	B1	PREMIUM
	5	B2	PREMIUM
★	NULL	NULL	NULL

Query 3: Get all shows for a specific movie.

```
SELECT s.ShowID, s.ShowDate, s.ShowTime, s.Price,  
       sc.ScreenNumber, t.Name AS Theatre  
FROM ShowTable s  
JOIN Screen sc ON s.ScreenID = sc.ScreenID  
JOIN Theatre t ON sc.TheatreID = t.TheatreID  
WHERE s.MovieID = 1  
ORDER BY s.ShowDate, s.ShowTime;
```

	ShowID	ShowDate	ShowTime	Price	ScreenNumber	Theatre
▶	1	2026-04-01	18:00:00	250.00	1	PVR Cinemas

Query 4: View all bookings made by a customer.

```
SELECT b.BookingID, m.Title, s.ShowDate, s.ShowTime,
       b.TotalAmount, b.BookingDate
FROM Booking b
JOIN ShowTable s ON b.ShowID = s.ShowID
JOIN Movie m     ON s.MovieID = m.MovieID
WHERE b.UserID = 3
ORDER BY b.BookingDate DESC;
```

	BookingID	Title	ShowDate	ShowTime	TotalAmount	BookingDate
▶	2	Inception	2026-04-01	21:00:00	300.00	2026-03-30

Query 5: Total revenue generated per movie.

```
SELECT m.Title, SUM(b.TotalAmount) AS TotalRevenue
FROM Booking b
JOIN ShowTable s ON b.ShowID = s.ShowID
JOIN Movie m     ON s.MovieID = m.MovieID
GROUP BY m.MovieID, m.Title
ORDER BY TotalRevenue DESC;
```

	Title	TotalRevenue
▶	KGF Chapter 2	560.00
	Interstellar	500.00
	Inception	300.00

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc.,Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Function:

A function is a set of SQL statements that you can pass parameters into, and they perform some operation and return a single value. Functions are used to encapsulate reusable logic within the database.

Functions:

1. Function to calculate refund amount for a booking (80% of total amount).

```
DELIMITER $$
CREATE FUNCTION GetRefundAmount(p_BookingID INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_Amount DECIMAL(10,2);
    SELECT TotalAmount INTO v_Amount
    FROM Booking
    WHERE BookingID = p_BookingID;
    RETURN v_Amount * 0.80;
END $$
DELIMITER ;
```

Query -> SELECT GetRefundAmount(1) AS RefundAmount;

	RefundAmount
▶	400.00

2. Function to get total number of bookings made by a user.

```
DELIMITER $$
CREATE FUNCTION GetUserBookingCount(p_UserID INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_Count INT;
    SELECT COUNT(*) INTO v_Count
    FROM Booking
    WHERE UserID = p_UserID;
    RETURN v_Count;
END $$
DELIMITER ;
```

Query -> SELECT GetUserBookingCount(3) AS TotalBookings;

	TotalBookings
▶	1

3. Function to calculate GST tax amount (18%) for a booking.

```
DELIMITER $$
CREATE FUNCTION GetTaxAmount(p_BookingID INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_Amount DECIMAL(10,2);
    SELECT TotalAmount INTO v_Amount
    FROM Booking WHERE BookingID = p_BookingID;
    RETURN ROUND(v_Amount * 0.18, 2);
END $$
DELIMITER ;
```

Query -> SELECT GetTaxAmount(1) AS GSTAmount;

	GSTAmount
▶	90.00

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Stored Procedure:

A stored procedure is a set of pre-compiled SQL statements that can be reused and shared by multiple programs. It can access or modify data in a database and supports input and output parameters.

Procedures:

1. Procedure to register a new customer.

```
DELIMITER $$
CREATE PROCEDURE RegisterCustomer(
    IN p_Name      VARCHAR(50),
    IN p_Email     VARCHAR(50),
    IN p_Phone     VARCHAR(15),
    IN p_Password  VARCHAR(50)
)
BEGIN
    IF EXISTS (SELECT 1 FROM User WHERE Email = p_Email) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Email already registered.';
    ELSE
        INSERT INTO User(Name, Email, Phone, Password, Role)
        VALUES (p_Name, p_Email, p_Phone, p_Password,
        'CUSTOMER');      SELECT LAST_INSERT_ID() AS NewUserID;  END
    IF; END $$
DELIMITER ;
```

Query -> CALL RegisterCustomer('Rahul
Sharma','rahul@gmail.com','9876543210','rahul123');

	NewUserID
▶	5

2. Procedure to book a ticket and process payment automatically.

```
DELIMITER $$
CREATE PROCEDURE BookTicket(
    IN p_UserID      INT,
    IN p_ShowID      INT,
    IN p_TotalAmount DECIMAL(10,2),
    IN p_PaymentMode VARCHAR(20)
)
BEGIN
    DECLARE v_BookingID INT;
    DECLARE v_PaymentID INT;

    INSERT INTO Booking(BookingDate, TotalAmount, UserID, ShowID)
    VALUES (CURDATE(), p_TotalAmount, p_UserID, p_ShowID);
    SET v_BookingID = LAST_INSERT_ID();

    INSERT INTO Payment(Mode, Status, TransactionDate, BookingID)
    VALUES (p_PaymentMode, 'SUCCESS', CURDATE(), v_BookingID);
    SET v_PaymentID = LAST_INSERT_ID();

    INSERT INTO DigitalInvoice(InvoiceDate, TaxAmount, PaymentID)
    VALUES (CURDATE(), ROUND(p_TotalAmount * 0.18, 2), v_PaymentID);

    SELECT v_BookingID AS BookingID, v_PaymentID AS PaymentID;
END $$
DELIMITER ;
```

Query -> CALL BookTicket(3, 1, 500.00, 'UPI');

	BookingID	PaymentID
▶	4	4

3. Procedure to cancel a booking and process refund.

```
DELIMITER $$
CREATE PROCEDURE CancelBooking(
    IN p_BookingID INT,
    IN p_Reason     VARCHAR(200)
)
BEGIN
    DECLARE v_Refund DECIMAL(10,2);
    SET v_Refund = GetRefundAmount(p_BookingID);

    INSERT INTO Cancellation(
        CancellationDate, Reason, RefundAmount, BookingID)
    VALUES (CURDATE(), p_Reason, v_Refund, p_BookingID);

    SELECT v_Refund AS RefundAmount;
END $$
DELIMITER ;
```

Query -> CALL CancelBooking(1, 'Change of plans');

	RefundAmount
▶	400.00

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Trigger:

A trigger is a set of SQL statements in the database which automatically execute whenever a special event occurs in the database, such as INSERT, DELETE, or UPDATE. Triggers help maintain data consistency automatically.

Triggers:

1. Trigger to automatically mark a seat as BOOKED after a booking is inserted.

```
DELIMITER $$
CREATE TRIGGER trg_AfterBookingInsert
AFTER INSERT ON Booking
FOR EACH ROW
BEGIN
    UPDATE Seat
    SET Status = 'BOOKED'
    WHERE ScreenID = (
        SELECT ScreenID FROM ShowTable WHERE ShowID = NEW.ShowID
    )
    AND Status = 'AVAILABLE'
    LIMIT 1;
END $$
DELIMITER ;
```

Query -> INSERT INTO Booking(BookingDate, TotalAmount, UserID, ShowID)
VALUES (CURDATE(), 500.00, 3, 1);

Seat status updated in Seat table.

	SeatID	SeatNumber	SeatType	ScreenID	Status
▶	1	A1	STANDARD	1	BOOKED
	2	A2	STANDARD	1	AVAILABLE
	3	A3	STANDARD	1	AVAILABLE
	4	B1	PREMIUM	1	AVAILABLE
	5	B2	PREMIUM	1	AVAILABLE
	6	A1	STANDARD	2	AVAILABLE

2. Trigger to prevent booking if no seats are available for a show.

```
DELIMITER $$
CREATE TRIGGER trg_BeforeBookingInsert
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    DECLARE v_Available INT;

    SELECT COUNT(*) INTO v_Available
    FROM Seat
    WHERE ScreenID = (
```

```

    SELECT ScreenID FROM ShowTable WHERE ShowID = NEW.ShowID
)
AND Status = 'AVAILABLE';

IF v_Available = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'No seats available for this show.'; END IF; END $$
DELIMITER ;

```

This trigger fires before every booking insert. If no available seats exist for the show's screen, it raises an error and prevents the booking from being inserted, maintaining data integrity.

3. Trigger to auto-generate DigitalInvoice after a successful payment.

```

DELIMITER $$
CREATE TRIGGER trg_AfterPaymentInsert
AFTER INSERT ON Payment
FOR EACH ROW
BEGIN
    IF NEW.Status = 'SUCCESS' THEN
        INSERT INTO DigitalInvoice(InvoiceDate, TaxAmount, PaymentID)
            SELECT CURDATE(),
                ROUND(b.TotalAmount * 0.18, 2),
                NEW.PaymentID
            FROM Booking b
            WHERE b.BookingID = NEW.BookingID;
    END IF;
END $$
DELIMITER ;

```

Query -> INSERT INTO Payment(Mode, Status, TransactionDate, BookingID)
VALUES ('UPI', 'SUCCESS', CURDATE(), 1);

Invoice automatically generated in DigitalInvoice table.

	InvoiceID	InvoiceDate	TaxAmount	PaymentID
▶	1	2026-03-30	90.00	1
	2	2026-03-30	54.00	2
	3	2026-03-30	100.80	3
	4	2026-03-31	90.00	4
	5	2026-03-31	90.00	5

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

4. Trigger to restore seat availability when a booking is cancelled.

```
DELIMITER $$
CREATE TRIGGER trg_AfterCancellationInsert
AFTER INSERT ON Cancellation
FOR EACH ROW
BEGIN
    UPDATE Seat
    SET Status = 'AVAILABLE'
    WHERE ScreenID = (
        SELECT s.ScreenID
        FROM Booking b
        JOIN ShowTable s ON b.ShowID = s.ShowID
        WHERE b.BookingID = NEW.BookingID
    )
    AND Status = 'BOOKED'
    LIMIT 1;
END $$
DELIMITER ;
```

Query -> INSERT INTO Cancellation(CancellationDate, Reason, RefundAmount, BookingID)
VALUES (CURDATE(), 'Wrong show time', 400.00, 1);

Seat status restored to AVAILABLE in Seat table.

	SeatID	SeatNumber	SeatType	ScreenID	Status
▶	1	A1	STANDARD	1	AVAILABLE
	2	A2	STANDARD	1	AVAILABLE
	3	A3	STANDARD	1	AVAILABLE
	4	B1	PREMIUM	1	AVAILABLE
	5	B2	PREMIUM	1	AVAILABLE
	6	A1	STANDARD	2	AVAILABLE

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re - accredited by NAAC with 'A' Grade

Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

References:

- [1] “MySQL Workbench,” *MySQL Documentation*. [Online]. Available: <https://dev.mysql.com/downloads/workbench/>
- [2] “VS Code for Java Development,” *Microsoft Documentation*. [Online]. Available: <https://code.visualstudio.com>
- [3] “Draw.io - Flowchart Maker & Online Diagram Software,” *diagrams.net*. [Online]. Available: <https://app.diagrams.net/>
- [4] “Online Movie Ticket Booking System - Database Design Concepts,” *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org>
- [5] “Database Normalization,” *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org>
- [6] “MySQL Triggers and Stored Procedures,” *MySQL Documentation*. [Online]. Available: <https://dev.mysql.com/doc>